

Revisão de estrutura de árvores

(Atividade individual remota para o dia 28/09)

Nome: Kamile Dias de Oliveira

Disciplina: Estrutura de Dados II **Professora:** Profa. Kadidja Valéria

Modalidade: individual, remota e assíncrona **Tempo estimado:** 1 hora e 45 minutos

A atividade pode ser realizada integralmente de casa. Não é necessário comparecer à faculdade.

Objetivos de aprendizagem

- Retomar os conceitos de nó, raiz, pai, filho, folha, altura e percurso.
- Diferenciar as regras e aplicações dos tipos de árvores estudados.
- Relacionar analogias cotidianas às propriedades técnicas de cada estrutura.

Etapa 1 — Revisão bibliográfica (35 minutos)

Consulte as **referências bibliográficas indicadas no plano de ensino e os materiais disponibilizados na disciplina**. Inicie pelos conceitos básicos de árvore; depois estude árvore geral, árvore binária, árvore binária de busca (ABB), AVL, rubro-negra, B e B+.

Para cada estrutura, procure identificar: como os dados são organizados; qual propriedade deve ser mantida; como funcionam busca e inserção; que ajustes podem ocorrer após alterações; e em que contexto ela é útil. Redija com suas próprias palavras.

1. Árvore Geral

- **Organização e Propriedades:** Os dados são dispostos hierarquicamente em níveis. A principal regra mantida é a relação pai-filho e a ausência de ciclos.
- **Busca e Inserção:** A inserção e a busca dependem de percorrer os nós da árvore, sem uma regra de ordenação pré-definida dos valores contidos nos nós, ou seja, o caminho para fazer a operação depende de percorrer e verificar os nós individualmente até encontrar a posição ou valor desejado.
- **Ajustes:** A árvore geral não possui rotinas de reequilíbrio automático ou rotações, o que significa que sua altura pode crescer de forma desproporcional conforme a ordem em que os dados são inseridos, podendo deixa-la desbalanceada.
- **Aplicações:** É amplamente utilizada para representar sistemas de arquivos (pastas e subpastas), organogramas empresariais e estruturas hierárquicas de documentos (como tags HTML e XML).

2. Árvore Binária

- **Organização e Propriedades:** Numa árvore binária, os dados são organizados a partir de um nó principal (raiz). Cada nó pode ter no máximo dois filhos, geralmente denominados *filho da esquerda* e *filho da direita*. Cada filho representa a raiz de uma nova subárvore. Como a estrutura é hierárquica e não possui ciclos, existe apenas um caminho entre a raiz e qualquer outro nó.
- **Busca e Inserção:** A árvore binária simples não estabelece uma regra específica de inserção baseada no valor dos elementos, sendo que a forma de incluir um novo nó depende do algoritmo ou da aplicação utilizada. Por não possuir uma regra de ordenação entre os filhos esquerdo e direito, o armazenamento dos dados não segue um padrão rígido. Assim, para visitar ou localizar os elementos na estrutura, utilizam-se diferentes métodos de percurso:
 - **Pré-ordem (RED):** visita a Raiz, depois a subárvore da Esquerda e, por fim, a Direita.
 - **Em-ordem / Simétrica (ERD):** visita a Esquerda, a Raiz e a Direita.
 - **Pós-ordem (EDR):** visita a Esquerda, a Direita e, por fim, a Raiz.
- **Ajustes:** Uma árvore binária simples não possui, por definição, mecanismos automáticos de balanceamento ou rotações. Sua forma depende da maneira como os nós são organizados, podendo apresentar diferentes alturas e, em determinados casos, ficar completamente desbalanceada.
- **Aplicações:** Pode ser utilizada na representação de expressões aritméticas, árvores de decisão e outras estruturas hierárquicas. Também serve como base para diferentes tipos de árvores utilizadas em estruturas de dados.

3. Árvore Binária de Busca (ABB)

- **Organização e Propriedades:** É uma variação especializada da árvore binária onde os dados são armazenados de forma ordenada. A sua propriedade fundamental determina que, para qualquer nó X da árvore:
 - Todos os valores presentes na subárvore da esquerda são estritamente menores que o valor de X .
 - Todos os valores presentes na subárvore da direita são estritamente maiores (ou maiores/iguais, dependendo da implementação) que o valor de X .
- **Busca e Inserção:**
 - **Busca:** Começa na raiz e compara o valor procurado com o nó atual. Se for menor, segue para a esquerda; se for maior, segue para a direita. O desempenho depende da altura da árvore, podendo variar de $O(\log n)$ em uma árvore balanceada até $O(n)$ no pior caso.

- **Inserção:** O novo elemento é comparado desde a raiz seguindo a mesma regra (menor à esquerda, maior à direita) até encontrar uma posição vaga (ponteiro nulo) onde é anexado como um nó folha.
 - **Percurso importante:** Ao realizar um percurso Em-Ordem (Em-Ordem / Simétrica) numa ABB, os dados são visitados em ordem estritamente crescente.
- **Ajustes:** A ABB simples não possui mecanismos de reequilíbrio automático. Se os dados forem inseridos de forma ordenada (ex: 1, 2, 3, 4, 5), a árvore não realiza rotações e degenera numa estrutura linear (semelhante a uma lista encadeada).
 - **Aplicações:** É utilizada em sistemas onde se busca rápida ordenação e pesquisa de dados em memória principal, como dicionários simples, tabelas de símbolos de compiladores e algoritmos de ordenação baseados em árvores (*Tree Sort*).

4. Árvore AVL

- **Organização e Propriedades:** É uma Árvore Binária de Busca (ABB) auto balanceada. Sua principal característica é manter a árvore equilibrada por meio do Fator de Balanceamento (FB), calculado pela diferença entre a altura da subárvore esquerda e da direita: Em uma AVL, o FB de cada nó deve ser -1, 0 ou +1.
- Busca e Inserção:
 - **Busca:** Segue a mesma regra de uma ABB: valores menores ficam à esquerda e maiores à direita. Como a árvore é balanceada, a busca tem complexidade $O(\log n)$ no pior caso.
 - **Inserção:** O novo elemento é inserido como em uma ABB. Depois, a árvore verifica o balanceamento dos nós. Se algum apresentar $FB = -2$ ou $+2$, é necessário reequilibrá-lo.
- Ajustes: O reequilíbrio é feito por **rotações**, que podem ser:
 - **Simples:** rotação para a direita ou para a esquerda.
 - **Duplas:** rotação esquerda-direita ou direita-esquerda.

- **Aplicações:** É útil quando se precisa de buscas rápidas e previsíveis, especialmente em estruturas que realizam muitas consultas.

5. Árvore Rubro – Negra

- **Organização e Propriedades:** É uma Árvore Binária de Busca (ABB) auto-balanceada que utiliza as cores vermelho e preto para manter a árvore equilibrada. A raiz é preta, as folhas NIL são consideradas pretas, um nó vermelho não pode ter um filho vermelho e todos os caminhos até as folhas devem possuir a mesma quantidade de nós pretos.
- **Busca e Inserção:** A busca segue a regra de uma ABB, com valores menores à esquerda e maiores à direita, apresentando complexidade $O(\log n)$ no pior caso. Na inserção, o novo nó é inicialmente vermelho. Caso a inserção viole alguma das regras da árvore, são realizados ajustes.
- **Ajustes:** O equilíbrio é mantido principalmente por recoloração e rotações. A recoloração altera as cores dos nós para corrigir conflitos, enquanto as rotações reorganizam a estrutura da árvore sem perder a ordenação dos valores.
- **Aplicações:** É muito utilizada quando é necessário realizar buscas, inserções e remoções eficientes, como em `std::map` e `std::set` do C++ e `TreeMap` do Java.

6. Árvore B

- **Organização e Propriedades:** É uma árvore de busca auto-balanceada e não binária, projetada para trabalhar com grandes volumes de dados, principalmente em dispositivos de armazenamento, como discos e SSDs. Cada nó pode conter várias chaves e vários filhos, definidos pela ordem da árvore. As chaves dentro de cada nó são mantidas ordenadas, todas as folhas ficam no mesmo nível e um nó com k chaves possui $k+1$ filhos.
- **Busca e Inserção:**
 - **Busca:** Realiza-se uma busca binária ou linear dentro das chaves do nó atual para determinar por qual dos ponteiros de filho o algoritmo deve descer, reduzindo drasticamente o número de acessos ao disco.
 - **Inserção:** O novo elemento é sempre inserido num **nó folha**. Se a folha ultrapassar limite máximo de chaves permitido pela ordem m , ocorre uma divisão.
- **Ajustes (Cisão / Split e Fusão / Merge):**

- **Cisão (Split):** Quando um nó fica cheio, ele é dividido em dois nós e a chave mediana "sobe" para ser promovida ao nó pai. Se a raiz for dividida, a altura da árvore aumenta em 1 nível (a árvore cresce de baixo para cima).
 - **Fusão (Merge):** Se uma remoção deixar um nó com menos chaves do que o mínimo permitido, ele pode redistribuir chaves com os vizinhos ou fundir-se com um nó irmão.
- **Aplicações:** É utilizada principalmente em bancos de dados e sistemas de armazenamento, especialmente quando é necessário trabalhar com grandes volumes de dados e reduzir o número de acessos ao armazenamento secundário.

7. Árvore B+

- **Organização e Propriedades:** É uma variação da Árvore B, muito utilizada para armazenar e organizar grandes volumes de dados. Os dados ficam somente nos nós folha, enquanto os nós internos armazenam apenas chaves que ajudam na navegação. Todas as folhas ficam no mesmo nível e são ligadas entre si, formando uma sequência que facilita o acesso aos dados.
- **Busca e Inserção:** Na busca, os nós internos são utilizados para encontrar a folha que contém o dado, com complexidade $O(\log n)$. Para buscas por intervalo, como encontrar todos os valores entre dois limites, é possível localizar a primeira folha e percorrer as folhas seguintes pela ligação entre elas. Na inserção, o elemento é colocado na folha correspondente. Se ela ficar cheia, ocorre uma divisão (split) e o índice é atualizado.
- **Ajustes:** Quando um nó fica cheio, ele pode ser dividido em dois, reorganizando as chaves e atualizando o nó pai. Na remoção, podem ocorrer redistribuição ou fusão de nós para manter a árvore balanceada.
- **Aplicações:** É muito utilizada em bancos de dados e sistemas de armazenamento, principalmente em índices que precisam realizar buscas eficientes e consultas por intervalo.

Etapa 2 — Quadro comparativo (35 minutos)

Preencha todas as linhas pertinentes ao conteúdo da disciplina. Acrescente árvore geral, árvore binária, heap e trie quando forem abordadas nas referências ou aulas. A coluna de referência deve apontar a fonte específica usada para cada linha.

Estrutura	Organização dos dados	Regra ou propriedade	Operação ou ajuste	Aplicação	Referência
Árvore Geral	Nós dispostos em estrutura hierárquica por níveis, onde cada nó pode ter múltiplos filhos	Relação pai-filho e ausência de ciclos; apenas a raiz não possui pai.	Busca e inserção requerem percorrer nós individualmente; sem reequilíbrio automático ou rotações	Sistemas de arquivos (pastas e subpastas), organogramas e marcas HTML/XML.	VETORAZZO, 2018; DROZDEK, 2018.
ABB	Nós organizados por ordenação de valores com no máximo dois filhos cada.	Subárvore esquerda contém apenas valores menores que o nó; subárvore direita contém apenas maiores (ou iguais).	Busca direcionada por comparação; sem reequilíbrio automático, podendo degenerar em lista encadeada.	Dicionários simples, tabelas de símbolos de compiladores e ordenação <i>Tree Sort</i>	DROZDEK, 2018; CELES & CERQUEIRA, 2004. VETORAZZO, 2018.
AVL	ABB auto-balanceada em termos de altura.	Fator de Balanceamento de cada nó deve manter-se estritamente em $-1,0$ ou $+1$.	Reequilíbrio através de rotações simples (esquerda/direita) ou duplas (esquerda-direita/direita-esquerda)	Estruturas de dados com foco em consultas e buscas rápidas e previsíveis.	DROZDEK, 2018.
Rubro-negra	ABB auto-balanceada baseada em coloração dos nós	Raiz é preta, folhas NIL são pretas, nó vermelho não tem filho vermelho e todos os caminhos têm a mesma quantidade de nós pretos.	Reequilíbrio realizado por recoloração de nós e rotações para resolver conflitos de cores.	Bibliotecas e sistemas que utilizam estruturas de busca ordenadas.	FRANCESQUINI (UNICAMP)
B	Árvore de busca auto-balanceada e não binária para memória secundária	Cada nó contém várias chaves ordenadas e $k + 1$ filhos; todas as folhas estão no mesmo nível	Inserção nas folhas com divisão (<i>split</i>) ao encher; remoção com fusão (<i>merge</i>) ou redistribuição	Bancos de dados e sistemas de arquivos em disco/SSD para reduzir acessos ao armazenamento.	DROZDEK, 2018.

B+	Variação da Árvore B onde dados reais residem apenas nas folhas.	Nós internos atuam apenas como índice; folhas ficam no mesmo nível e estão interligadas em sequência.	Busca ponto a ponto no índice; busca por intervalo diretamente na ligação entre as folhas	Índices de bancos de dados e sistemas de arquivos com consultas frequentes por intervalo	DROZDEK, 2018.
Heap	Árvore binária quase completa armazenada sequencialmente em vetor.	Numa Heap Max, o pai é sempre maior ou igual aos filhos (numa Heap Min, é menor ou igual).	Inserção posiciona no fim e promove o nó (<i>percolate up</i>); remoção retira do topo e ajusta (<i>percolate down</i>).	Filas de prioridade, ordenação <i>Heapsort</i> e gerenciamento de processos no sistema operacional.	DROZDEK, 2018;
Trie	Árvore de busca digital/prefixada baseada em caracteres.	Nós representam caracteres de chaves; prefixos comuns são compartilhados pelo mesmo caminho.	Busca e inserção percorrem os caracteres da chave individualmente de forma sequencial.	Autocompletar texto, verificadores ortográficos e roteamento de endereços IP.	DROZDEK, 2018.

Etapa 3 — Identificação por analogias (35 minutos)

Para **cada situação**, identifique a estrutura, justifique sua resposta com uma propriedade técnica e explique **um limite da analogia** (algo que a comparação não representa fielmente). Evite responder apenas com o nome da árvore.

- Uma estante de números é reorganizada por rotações quando um lado fica alto demais em relação ao outro.
 - **Estrutura:** Árvore AVL.
 - **Justificativa Técnica:** A AVL utiliza o Fator de Balanceamento (FB), que compara a altura das subárvores esquerda e direita. Quando a diferença ficam fora de -1, 0 ou +1, são realizadas rotações para reequilibrar a árvore.
 - **Limite da Analogia:** A reorganização de uma estante muda fisicamente a posição dos objetos, enquanto a rotação em uma AVL altera as relações entre os nós, mantendo a ordem dos valores.
- Um catálogo guarda várias chaves por página; quando uma página fica cheia, ela é dividida.
 - **Estrutura:** Árvore B.
 - **Justificativa Técnica:** Cada nó da Árvore B pode armazenar várias chaves ordenadas. Quando um nó fica cheio, ele é dividido (*split*) e uma chave é promovida para o nó pai.

- **Limite da Analogia:** No catálogo físico, uma página precisa ser reorganizada manualmente, enquanto na Árvore B a divisão dos nós é realizada pelo próprio algoritmo.

3. Uma fila mantém a tarefa de maior prioridade no topo para retirá-la primeiro.

- **Estrutura:** Heap (Heap Máxima / Fila de Prioridades).
- **Justificativa Técnica:** A propriedade estrutural da Heap Máxima garante que o valor de qualquer nó pai seja maior ou igual ao de seus filhos, fazendo com que o elemento de maior valor (maior prioridade) esteja sempre armazenado diretamente na raiz e possa ser acessado diretamente.
- **Limite da Analogia:** Uma fila real é organizada de forma linear, enquanto a Heap possui uma estrutura hierárquica e geralmente é armazenada em um vetor.

4. Um índice percorre letras sucessivas e compartilha o início das palavras de mesmo prefixo.

- **Estrutura:** Trie (Árvore de Prefixos).
- **Justificativa Técnica:** A Trie organiza as palavras caractere por caractere. Palavras que possuem o mesmo prefixo compartilham os mesmos nós no início do caminho.
- **Limite da Analogia:** Um índice de livro normalmente apresenta palavras completas e páginas, enquanto a Trie representa as palavras por meio de caminhos formados pelos caracteres.

5. Uma estrutura usa cores, recolorações e rotações para manter controlada a altura dos caminhos de busca.

- **Estrutura:** Árvore Rubro-Negra (Red-Black Tree).
- **Justificativa Técnica:** Atribui uma flag de cor (Vermelho ou Preto) a cada nó e impõe regras estritas (como raiz preta, ausência de nós vermelhos consecutivos e quantidade uniforme de nós pretos até as folhas). Quando a inserção viola essas regras, a árvore aplica recoloração e rotações para reestabelecer o equilíbrio aproximado da altura.
- **Limite da Analogia:** A cor na estrutura de dados é apenas uma informação binária (bit ou booleano) usada pelo algoritmo de decisão, sem qualquer relação com aspectos visuais ou estéticos do mundo físico.

6. Um índice conduz às folhas que contêm os registros, ligadas entre si para facilitar consultas por intervalo.

- **Estrutura:** Árvore B+.
- **Justificativa Técnica:** Na B+, os nós internos funcionam como índices, enquanto os dados ficam nas folhas. As folhas são ligadas entre si, facilitando a busca e a leitura de valores em um determinado intervalo.
- **Limite da Analogia:** Um índice físico exige consultar as páginas indicadas, enquanto a B+ permite percorrer diretamente as folhas depois que o primeiro ponto do intervalo é encontrado.

7. Numa coleção de números, cada nó direciona valores menores para a esquerda e maiores para a direita.

- **Estrutura:** Árvore Binária de Busca (ABB).
- **Justificativa Técnica:** Na ABB, os valores menores que um nó ficam na subárvore esquerda e os maiores ficam na subárvore direita, permitindo direcionar a busca de acordo com o valor pesquisado.
- **Limite da Analogia:** Direcionar objetos numa coleção física exige mover caixas ou recipientes no espaço tridimensional, enquanto na ABB o direcionamento representa apenas uma avaliação condicional (se valor < no.valor) que determina qual ponteiro de memória seguir durante a navegação.

Referências

CELES, Waldemar; CERQUEIRA, Renato; RANGEL, José Lucas. **Introdução a Estruturas de Dados: com técnicas de programação em C**. Rio de Janeiro: Elsevier / Campus, 2004.

CORMEN, Thomas H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

DROZDEK, Adam. **Estrutura de Dados e Algoritmos em C++**. Tradução da 4. ed. norte-americana. São Paulo: Cengage Learning, 2018.

VETORAZZO, Adriana de Souza. **Estrutura de Dados**. Porto Alegre: SAGAH, 2018.

FRANCESQUINI, Emilio. **MC202: Estruturas de Dados – Árvores Binárias de Busca e Árvores Balanceadas**. Instituto de Computação, Universidade Estadual de Campinas (UNICAMP). Notas de aula. Disponível em: <https://www.ic.unicamp.br/~francesquini/mc202/files/aula16-18.pdf>. Acesso em: 28 set. 2026.